Pacing Gain and Congestion-Window Growth: Packet Models and SCP Experiments

Skai Uijing
Independent Researcher
China
skaiuijing@gmail.com

Abstract

Pacing changes when new congestion-window credit becomes a transmission and, consequently, when it produces an ACK. Motivated by slower startup in SCP, we express slow-start growth through actual transmissions, construct Fibonacci and immediate-send comparisons, and solve a packet scheduler with immutable reservations. Its asymptotic window multiplier is $\min(e^{g/2}, 2)$, with a transition at $g = 2\ln 2$. Two campaigns execute the actual SCP C core over a loss-free, fixed-100-ms-feedback virtual network: each has 66 fixed-gain transfers and three native-policy transfers. Eight low slow-start gains match the predicted late multiplier within 0.003%. Retained next-slot scheduling brings packet-group spans close to the ideal model, while execution precision still changes finite trajectories. In congestion avoidance, integer-byte updates produce a 15.719% full-packet window deficit against the real-valued ACK recurrence. We separate these native observations from an independent Python model and from mathematical predictions, and distinguish window growth, ACK-indexed growth, and transmission dispersion.

CCS Concepts

• **Networks** → **Transport protocols**.

Keywords

pacing, slow start, congestion avoidance, congestion window, virtual-network experiments

1 Introduction

While building SCP with Reno-style congestion control, I first used burst sending and then added pacing. Transfers slowed down even though the ratio of useful data to total transmitted data barely changed. Tracing packets and ACKs suggested a timing question: an ACK can enlarge the window immediately, but newly permitted packets still have to pass through the pacer before they produce feedback.

For example, send one packet at time zero. Its ACK returns at R, increasing the window to two packets. If the next packets leave at R and $1.5R$, only one has been acknowledged at $2R$: the window is three, not four. All window updates occur on ACK arrival. This example motivates the analysis; it does not describe every pacer or imply that pacing generally harms network performance.

We also isolate loss-free congestion avoidance from one MSS to examine how its smaller ACK increments and integer-byte storage change this timing problem.

2 ACK accounting and two comparison models

Assume a fixed send-to-ACK delay R, no loss, continuously available data, one ACK per full-sized packet, and indefinite slow start. Each ACK adds one MSS, denoted M, to the window. Normalize time and window as $\tau = t/R$ and $w = \mathrm{cwnd}/M$. With actual packet-send times s_j in normalized units, including $s_0 = 0$,

$$\begin{aligned} w(\tau) &= 1 + \sum_{j\ge 0} \mathbf{1}_{\{s_j+1\le\tau\}}, \\ W_{n+2} &= W_{n+1} + S_n. \end{aligned} \tag{1}$$

Here $W_n = w(n)$ is sampled after all boundary events, and S_n counts sends in $(n, n+1]$. Sampling at RTT boundaries does not defer ACK processing.

Immediate sending. ACKs in $(n, n+1]$ can only acknowledge packets already in flight just after n. If their number is $\mathcal{F}_n$, then $W_{n+1} - W_n \le \mathcal{F}_n \le W_n$. Thus $W_n \le 2^n$. Sending both newly permitted packets immediately at every ACK attains $\mathrm{cwnd}(nR) = M2^n$ under these ideal assumptions.

Fixed batches. A separate comparator reads its window V_n at each boundary and sends at $n + i/V_n$, for $i = 0, \ldots, V_n - 1$. ACKs update the window immediately but leave that batch unchanged. The next boundary supplies the next batch's first packet, so $S_n = V_n$ and

$$\begin{aligned} V_{n+2} &= V_{n+1} + V_n, \quad V_0 = 1,\ V_1 = 2, \\ \mathrm{cwnd}_{\mathrm{fixed}}(nR) &= MF_{n+2} = \frac{M(\phi^{n+2} - \psi^{n+2})}{\sqrt{5}}. \end{aligned} \tag{2}$$

where $\phi = (1+\sqrt{5})/2$, $\psi = (1-\sqrt{5})/2$, and $F_0 = 0, F_1 = 1$. This gives $1, 2, 3, 5, 8, 13, \ldots$. It is a deliberately simplified comparator. For any scheduler satisfying the additional service condition $S_n \ge W_n$, induction in (1) gives $MF_{n+2} \le \mathrm{cwnd}(nR) \le M2^n$. The lower condition requires justification for the scheduler being studied; earlier reservation alone is not a proof. Appendix A proves the lower comparison for the specific queued reference below by comparing actual packet times. It is not assumed for every pacer. These bounds motivate exponential growth but do not establish an exact exponential solution.

3 An exact queued reference model

3.1 Reservations, flight, and feedback

Consider the following particular scheduler, with a fixed gain $g \ge 1$. After ACK k, the window is $k+1$. The ACK releases one flight position and adds one window position, permitting two new packets. Both are appended to the reservation queue with spacing $1/\lfloor g(k+1)\rfloor$; existing timestamps remain unchanged. Let a_k be ACK time, b_k the new batch's first send, and T_k the next available slot after

this batch. Then

$$\begin{aligned} a_k &= s_{k-1} + 1, & b_k &= \max(a_k, T_{k-1}), \\ s_{2k-1} &= b_k, & s_{2k} &= b_k + \frac{1}{g(k+1)}, \\ T_k &= b_k + \frac{2}{g(k+1)}. \end{aligned} \tag{3}$$

Initialize $s_0 = 0$, $T_0 = 1/g$. Actual sends move packets from the reserved queue to flight. Consequently $F+Q = w$ after each ACK's reservations, and actual flight respects the window. This model retains the spacing after the last reserved packet, even if the queue becomes empty. The tested SCP implementation now uses this same next-slot rule; Section 4 specifies its integer execution.

3.2 Solving the event recurrence

Write $c = 2/g$, $H_k = \sum_{i=1}^{k} 1/i$, and $D(q) = H_{2q} - H_q$. Group ACK indices as $q \le k < 2q$, where $q = 2^m$, and put $B_m = b_{2^m}$. These are analytical groups, not batches processed by the implementation. The exact solution is

$$B_{m+1} = B_m + \max\{1, cD(2^m)\}, \quad B_0 = 1, \tag{4}$$

$$b_k = B_m + c(H_k - H_{2^m}), \quad 2^m \le k < 2^{m+1}. \tag{5}$$

To verify it, note that $b_k = \max(a_k, b_{k-1} + c/k)$ for $k \ge 2$, and $a_{2j} = b_j + 1$, $a_{2j+1} = b_j + 1 + c/[2(j+1)]$. At the first index q of a group, the two candidate times are $B_{m-1} + 1$ and $B_{m-1} + cD(q/2)$, proving (4). Set $\Delta = B_m - B_{m-1} \ge 1$. At interior even indices,

$$T_{2j-1} - a_{2j} = \Delta - 1 + c[D(j) - D(q/2)] > 0. \tag{6}$$

At interior odd indices the bracket additionally contains $1/(2j+1) - 1/(2j+2) > 0$. Since $D(j+1) - D(j) = 1/[(2j+1)(2j+2)] > 0$, the queue branch wins throughout each group's interior. Summing its increments proves (5) by induction. In particular,

$$b_k = cH_k + 1 - c + \sum_{i=0}^{\lfloor \log_2 k \rfloor - 1} \left[1 - cD(2^i)\right]_+ . \tag{7}$$

Here $[x]_+ = \max(x, 0)$. Together with (3) and (1), this determines the window at every time without fitting a growth curve.

3.3 Gain, finite spans, and the limiting multiplier

Group m contains packets $2q-1, \ldots, 4q-2$. Its first-to-last send span Λ_m and extra ACK waiting time I_m, excluding the nominal trailing spacing, are

$$\frac{\Lambda_m}{R} = \frac{2D(q) - 1/(2q)}{g}, \qquad \frac{I_m}{R} = \left[1 - \frac{2D(q)}{g}\right]_+ . \tag{8}$$

Because $D(q) \uparrow \ln 2$, the limiting transition is $g_c = 2\ln 2 \approx 1.38629436112$. Below it, extra waiting eventually ends, $b_k = (2/g)H_k + \text{constant}$, and $w(\tau) = A_g e^{g\tau/2} + O(1)$. At or above it, $B_m = m + 1$. Counting sends gives, for integer $n \ge 1$ and $0 \le u < 1$,

$$\begin{aligned} w(n+u) &= 2^{n-1} \min(e^{gu/2}, 2) + O_g(1), \\ r(g) &:= \lim_{n\to\infty} \frac{W_{n+1}}{W_n} = \min(e^{g/2}, 2). \end{aligned} \tag{9}$$

For finite $g \ge g_c$, the boundary value is exactly $W_n = 1 + 2^{n-1}$, rather than 2^n. The group span approaches $\Lambda_m/R \to g_c/g$: higher gain compresses the active part of each group. This is an asymptotic statement about packet-index groups, not a claim that every fixed RTT interval has this span.

4 From the reference model to an SCP experiment

4.1 What an instantaneous-rate approximation predicts

A different idealization updates the entire sending rate continuously: in normalized units $x(\tau) = gw(\tau)$, while window credit remains available. Fixed feedback gives $w'(\tau) = x(\tau - 1) = gw(\tau - 1)$. Substitution of $w = Ae^{\lambda\tau}$ yields

$$\begin{aligned} \lambda e^{\lambda} &= g, \\ F(\tau) &= \int_{\tau-1}^{\tau} x(v)\, dv = (e^{\lambda} - 1) w(\tau). \end{aligned} \tag{10}$$

The flight constraint requires $e^{\lambda} \le 2$, reaching equality at $g = 2\ln 2$. Thus the two idealizations share a transition value but differ below it. At $g = 1$, the instantaneous-rate multiplier is approximately 1.763223, while the immutable-queue multiplier is $\sqrt{e} \approx 1.648721$. Previously reserved packets retain older, wider spacings in the latter model. An exponential ansatz also fixes a compatible history; it does not determine the amplitude generated by a single initial packet.

4.2 Integer scheduling in the tested core

The experiment executes the SCP C sender, receiver, pacing callback, and ACK processing. The virtual network transports their packets; it does not compute the window from a recurrence. The core stores reservation times in unsigned 64-bit Q32.32 milliseconds. Each congestion controller supplies a Q16 gain K. For a fixed target g, payload length ℓ, measured RTT R_{ms}, and byte window C, scheduling uses

$$\begin{aligned} d_{\text{Q32}} &= \left\lfloor \frac{\ell R_{\text{ms}} G}{C} \right\rfloor, \\ K &= \text{round}(2^{16} g), \quad G = \left\lfloor 2^{48}/K \right\rfloor . \end{aligned} \tag{11}$$

Write $g_Q = K/2^{16}$; reciprocal rounding adds the smaller difference between g_Q and $2^{32}/G$. At each output call the next slot becomes $\max(T, \text{now})$. A packet receives that timestamp, then advances T by its own spacing. Fractional parts and the trailing slot survive an empty queue. Thus a packet constrains the next packet's earliest slot. The send routine uses the reservation's integer part: a reservation at 1.8 ms is eligible at 1 ms. The virtual clock advances by 1 ms, allowing multiple sends per tick. ACK-triggered scheduling and the pacing timer share the due-send routine.

The absence of SRTT uses a 1 ms fallback. Once feedback is available, the measured RTT is 100 ms. An independent Python model uses the same next-slot rule but a 1 μs clock, truncates each spacing to whole microseconds, and uses the unquantized target gain. Its CA window is real-valued. These are declared differences, not interchangeable implementations: native Q32 accumulation retains fractions that the Python model discards. Python sends only full

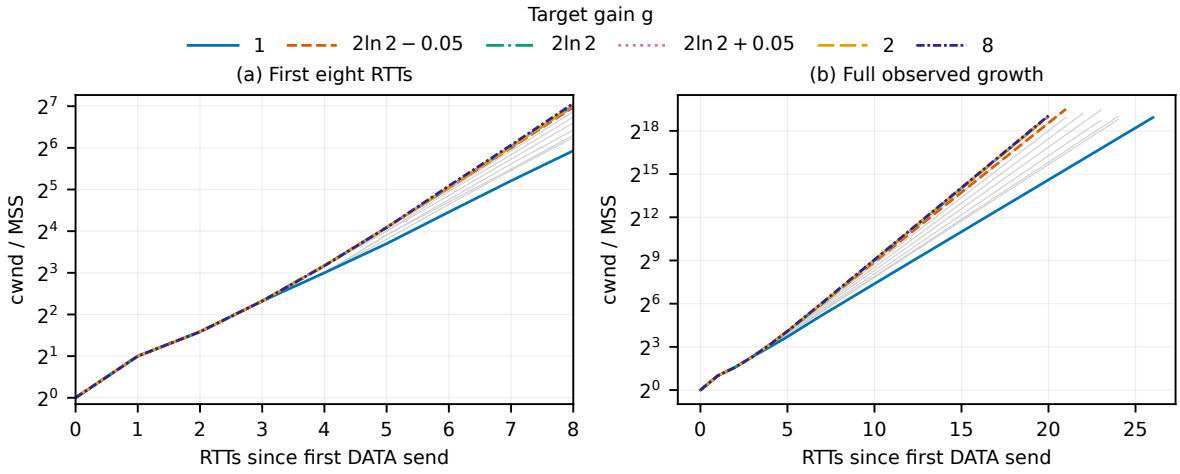


Figure 1: Actual SCP slow-start windows at RTT boundaries: all 22 gains in gray, six representative gains highlighted, with early and full-growth views. Time is measured from the first DATA send in units of the 100 ms feedback RTT; windows are in MSS on a base-two logarithmic axis. Lines connect samples.

Table 1: SCP virtual-network experiment configuration.

Quantity	Configuration
Data and packet size	1 GiB generated data; MSS 1430 bytes; final payload 584 bytes
Window and mode	Initial window 1 MSS; separate sustained SS and CA campaigns
Network	50 ms per direction; no loss, rate cap, or serialization delay
Feedback RTT	100 ms from actual DATA send to returning ACK input
Observation	Core state every 1 ms; actual DATA and ACK events logged
Gains	$g_c + 0.05k$, $k = -6, \ldots, 6$; 1, 1.1, 1.75, 2, 2.5, 3, 4, 6, 8
Repetitions	Per phase: 22 fixed gains × 3, plus native policy × 3
Limits	cwnd cap $2^{31} - 1$ bytes; buffer/window settings 1 GiB

MSS packets, totaling 1 GiB plus 846 bytes; native SCP sends exactly 1 GiB with a short final packet.

4.3 Configuration and verification

All 138 native transfers completed with 750,869 DATA packets each and matching endpoint checksums. No loss, retransmission, or cwnd-cap contact occurred. Fifteen functional tests passed in each campaign. The shared core, library, virtual network, and common-transfer files exactly matched baseline `e41ac88`. CA is selected at the research entry by setting ssthresh to one MSS, without a core patch. Audits follow every DATA send through the receiver's input and ACK generation to the sender's ACK input, verify 100 ms feedback and every window update, and check actual sends against reserved timestamps. Three repetitions per configuration produced identical raw traces.

At target g_c, $K = 90852$, $g_Q = 1.38629150390625$, and $G = 3098170394$. The Q16 value is slightly below g_c, so the exact $g \geq g_c$ theorem cannot simply be assigned to this finite run. The first DATA send defines time zero; the handshake is excluded. Repetitions are determinism checks, without random-error bars. Span summaries require complete, non-final-send intervals with at least two packets; all other intervals remain in the raw tables. Middle–late summaries use bins from 50% through 90% of the final-send bin index.

Table 2: Measured growth and completion for selected gains.

Target gain	W_{18}/W_{17}	W_{18} (MSS)	Last ACK (t/R)
1	1.648756	9,144	26.82
$g_c - 0.10$	1.902456	73,216	21.62
$g_c - 0.05$	1.950624	100,818	21.01
g_c	2.000000	131,984	20.51
2	2.000000	132,390	20.35
8	2.000000	136,422	20.08

5 Measured congestion-window growth

The low-gain regime. Figure 1 shows slower growth at low gain. For the eight sampled gains from 1 through $g_c - 0.05$, the measured boundary ratio W_{18}/W_{17} differs from $e^{g_Q/2}$ by less than 0.003%. At $g = 1$ the ratio is 1.648756, close to the queued reference $\sqrt{e}$ and distinct from the instantaneous-rate prediction 1.763223. This supports the queued model's multiplier in this regime, without asserting equality of every timestamp or window value.

Doubling does not remove the startup deficit. At the critical target, boundary windows begin 1, 2, 3, 5, 9, 17, 33, 65, 129 MSS. Later values include $W_{14} = 8249$, $W_{18} = 131984$, and $W_{19} = 263968$. The samples double from RTT 14 through 19, yet W_{18} is 49.6521% below the immediate-send reference 2^{18}. At RTT 20 the window is 527935 MSS, one MSS below another exact doubling. The finite

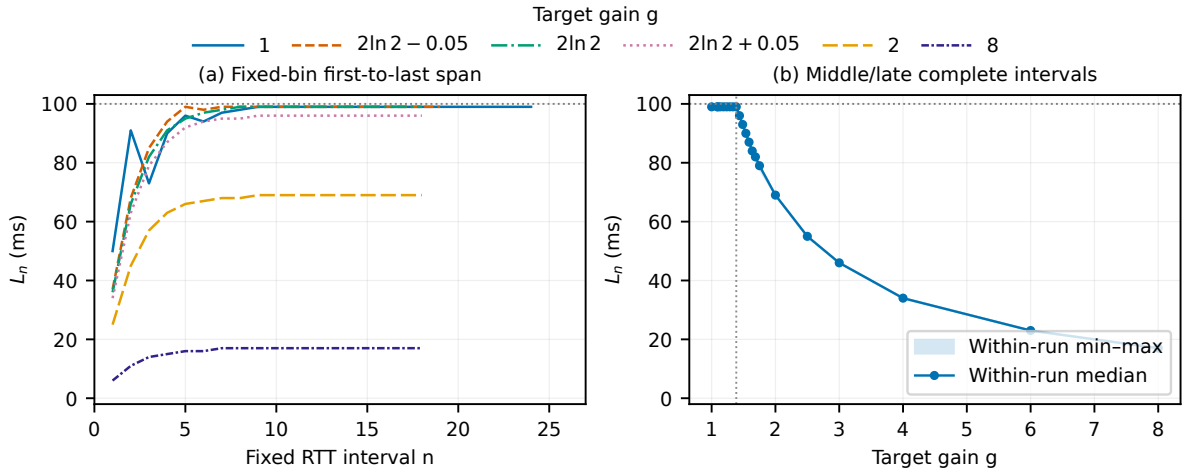


Figure 2: Actual SCP slow-start sending spans. Representative within-run curves are accompanied by the middle–late summary across all 22 gains. Incomplete, singleton, and final-send intervals are excluded from the summaries. Spans come from packet timestamps, not a nominal rate.

trace therefore supports a late multiplier near two without an exact infinite-time doubling or within-RTT periodicity claim.

6 Within-RTT sending span and concentration

For this figure, let $\mathcal{P}_n$ contain sends in the half-open interval $[nR, (n+1)R)$, measured from the first DATA transmission. Define

$$L_n = \max_{j\in\mathcal{P}_n} t_j - \min_{j\in\mathcal{P}_n} t_j. \tag{12}$$

A singleton has span zero; an empty interval has no defined span. This bin convention differs from the right-closed send-count intervals in (1). It also differs from the packet-index groups defining Λ_m in (8). These measurements cannot be interchanged near a phase boundary.

Evolution within one run. At the critical target, L_n begins $0, 36, 66, 82, 91, 95, 97, 98$ ms for $n = 0, \ldots, 7$, then remains 99 ms for $n = 8, \ldots, 18$. The final-send interval has span 51 ms and is excluded from aggregates. The finite millisecond plateau is distinct from the harmonic convergence of the exact reference group's span.

Variation across gains. At RTT 18, fixed-bin spans are 99 ms at the critical target, 69 ms at $g = 2$, and 17 ms at $g = 8$. A 100 ms half-open interval sampled on a 1 ms grid has a maximum span of 99 ms. That value means sends reach both edge bins; it neither proves uniform density nor excludes a packet-index group extending across an RTT boundary. In particular, low-gain groups can exceed one RTT while their fixed-bin spans remain 99 ms.

Why precision matters. With $R = 100$ ms, an ideal full-packet spacing is $100/(gw)$ ms. Native SCP retains its sub-millisecond fraction in the next slot. The independent Python model instead truncates every spacing to microseconds; at $w > 100000/g$ MSS, that spacing becomes zero. Thus a finer execution clock alone does not guarantee a closer approximation. At $g = 1$, Python completes at 25.21217 RTTs, native SCP at 26.82, and numerical evaluation of the ideal ACK-index scheduler at 26.82611. These discrepancies arise in separately labeled models; the Python results are not native measurements.

From the critical target to $g = 8$, the last DATA ACK moves from 2051 to 2008 ms, an improvement of 2.10%. Meanwhile L_{18} falls from 99 to 17 ms, and the largest millisecond bin's share of RTT-18 packets rises from 1.3813% to 7.7396%. Faster completion and more concentrated sending are distinct effects; span alone does not describe internal density.

7 Loss-free congestion avoidance

ACK count and elapsed time. In the real-valued full-packet form of RFC 5681, Section 3.1, equation (3) [1], let u_k be the window in MSS after CA ACK k, with $u_0 = a \geq 1$. Then

$$\begin{aligned} u_{k+1} &= u_k + \frac{1}{u_k}, \\ u_K^2 &= a^2 + 2K + \sum_{i=0}^{K-1} \frac{1}{u_i^2}. \end{aligned} \tag{13}$$

The identity follows by squaring each update. Since $u_i^2 \geq a^2 + 2i$, it gives $u_K^2 = a^2 + 2K + O(\ln(K+1))$. Gain does not enter this ACK-indexed recurrence. The window at time t is $Mu_{K(t)}$, where $K(t)$ counts actual returning ACKs; substituting t/R for K would discard the many ACKs within an RTT.

What the retained-tail model proves. Extend the ideal scheduler of Section 3 with whole-packet credit $F + Q = \lfloor u \rfloor$. After ACK k it reserves

$$\begin{aligned} m_k &= 1 + \lfloor u_k \rfloor - \lfloor u_{k-1} \rfloor, \\ T_k &= \max(a_k, T_{k-1}) + \frac{m_k}{gu_k}. \end{aligned} \tag{14}$$

Here time is normalized by R, a_k is the corresponding packet's send time plus one, and T_k retains the trailing interval. Put $\mathcal{I}_K =$

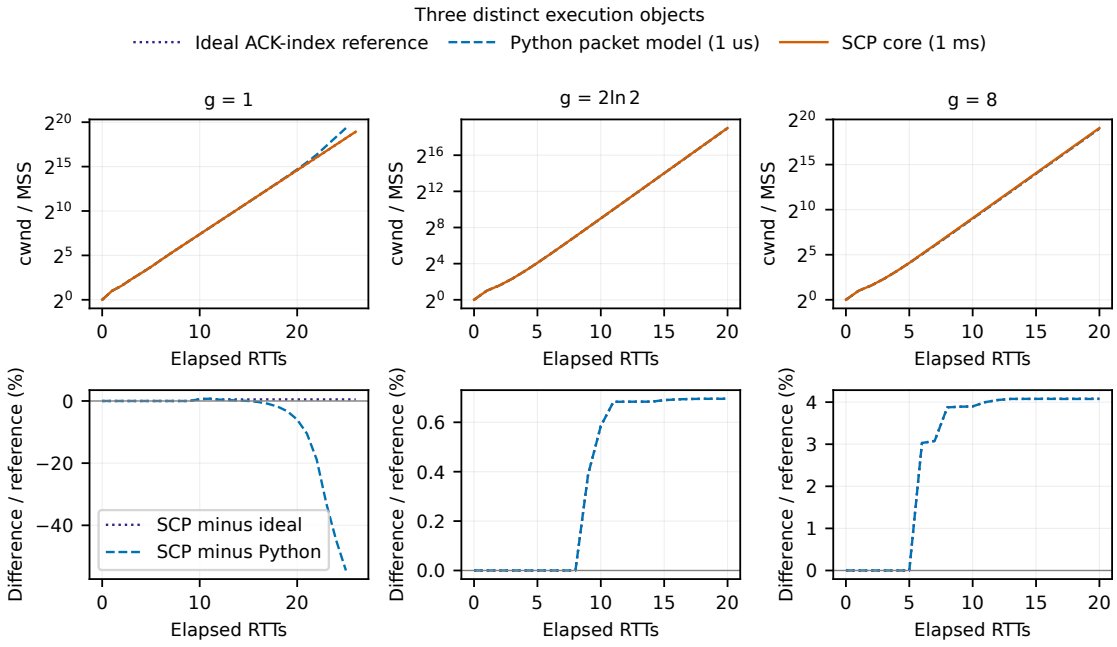


Figure 3: Three distinct SS execution models, at the same elapsed RTT boundaries. Top: native SCP, the independent Python packet model, and numerical evaluation of the ideal ACK-index recurrence. Bottom: native minus each reference, divided by that reference. No time shift or amplitude fit is applied. The large late $g = 1$ Python discrepancy exposes per-spacing truncation; jumps in pointwise error also depend on ACK phase.

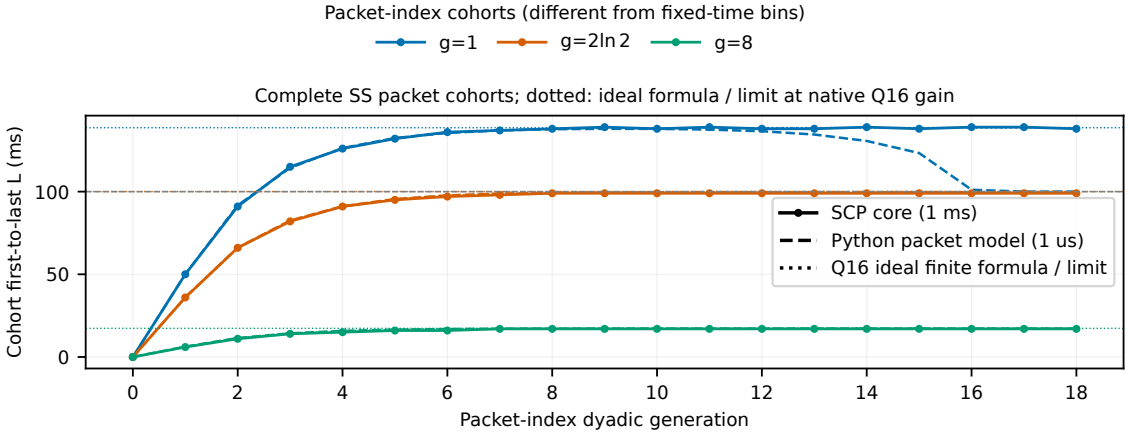


Figure 4: The same packet-index groups compared across native SCP, the independent Python model, and the finite harmonic reference (8) evaluated at the native Q16 gain. Dotted horizontal lines are its corresponding limits. Unlike fixed-bin spans, a group's span may exceed one RTT. Native millisecond execution and Python's per-spacing microsecond truncation remain visible.

$\sum_{k=1}^{K}[a_k - T_{k-1}]_+$. Telescoping (14) splits service into $\sum 1/u_k$ for replacement packets and $\sum(\lfloor u_k \rfloor - \lfloor u_{k-1} \rfloor)/u_k$ for extra whole packets. For fixed $g \geq 1$ and indefinitely continued CA,

$$T_K = \mathcal{I}_K + \frac{u_K + \ln u_K}{g} + C + O(1/u_K). \tag{15}$$

Indeed, $1/u_k = (u_k - u_{k-1}) - 1/(u_{k-1}^2 u_k)$ makes the first sum $u_K + C_1 + O(1/u_K)$. Each sufficiently large integer threshold n is crossed at a window $v_n = n + O(1/n)$, so the second sum is harmonic: $\ln u_K + C_2 + O(1/u_K)$. This proves (15), but does not determine $\mathcal{I}_K$ or a closed window law in time. It uses a real window and

Table 3: CA completion and within-run span ranges, $n = 100, \ldots, 1200$.

Target gain	Last ACK (t/R)	Span range (ms)
1	1341.81	99
g_c	1339.48	99
2	1339.31	71–72
4	1339.14	33–34
8	1339.07	16

ideal time; SCP's integer updates and execution grid still require separate examination.

Integer-byte updates in SCP. A new campaign uses Table 1's conditions but sets both initial cwnd and ssthresh to M. This artificial initial state isolates CA; it is not ordinary TCP startup. The research target changes threshold initialization while reusing the shared core, virtual network, and pacing rules. For an ACK covering N_k new bytes, the core uses

$$C_k = C_{k-1} + \max\left(1, \left\lfloor \frac{MN_k}{C_{k-1}} \right\rfloor\right), \qquad C_0 = M. \tag{16}$$

This differs from the real-valued recurrence, even for full-MSS ACKs. RFC 5681 also discusses the integer-zero increment and rounding it to one byte; its equation (3) is one allowed CA method, not every implementation of Reno. For full packets, once $C > M^2/2 = 1{,}022{,}450$ bytes, (16) adds exactly one byte per ACK. Discarded fractional increments accumulate into a measurable difference.

All 66 fixed-gain CA transfers and three native-policy transfers completed with matching checksums, exactly 100 ms feedback, no retransmission, and no cap contact. Every observed ACK update matched (16) with zero byte error. At full-packet ACK $K = 750868$, the actual window was 1,476,947 bytes, versus 1,752,403.2934 bytes from (13): measured minus reference, divided by reference, is −15.719%. The final 584-byte ACK then raises the actual window to 1,476,948 bytes and is excluded from this full-packet comparison.

At the same full-packet ACK count, the verified integer window sequence is identical across gains; gain changes when those ACKs occur. Among Figure 5's fixed gains, the largest RTT-boundary difference from gain 1 is 2.340 MSS. From $g = 1$ to 8, completion improves by only 0.204%, while dispersion changes substantially. Table 3 reports the full span range over 1101 intervals $n = 100, \ldots, 1200$, rather than assuming exact periodicity. These millisecond observations do not make $2 \ln 2$ a CA transition. At matching elapsed times, native windows differ from the independent real-window models by up to about 21.9%; this includes different ACK timing as well as integer updates, unlike the matching-ACK-count comparison above.

A continuous candidate for adaptive gain. Normalize rate to packets per RTT. With actual send rate $q(\tau)$, fixed feedback gives $w' = q(\tau - 1)/w$. The flight is therefore

$$\begin{aligned} f(\tau) &= \int_{\tau-1}^{\tau} q(s)\, ds \\ &= \frac{w(\tau+1)^2 - w(\tau)^2}{2}. \end{aligned} \tag{17}$$

Continuous equality $f = w$ requires

$$w(\tau+1) = \sqrt{w(\tau)^2 + 2w(\tau)}, \quad q = (w+1)w'.$$

On an additionally assumed smooth compatible growth branch, expansion gives $w' = 1 - 1/(2w) + O(w^{-2})$, hence

$$g_{\text{fill}} = q/w = 1 + \frac{1}{2w} + O(w^{-2}). \tag{18}$$

This is a first-order fluid candidate, not an exact whole-packet optimum or a proof that every interval has $L = R$. The native AIMD policy uses $K = 90852$ in SS and $K = 65536 + \lfloor M\,65536/(2C) \rfloor$ in CA. Its three additional CA runs complete at 1341.39 RTTs; gain falls from 1.5 to approximately 1.000473, with 99 ms spans throughout the 403 selected middle–late bins. The SS native policy matches its fixed-Q16 critical-target configuration. No matching dynamic-policy Python campaign is claimed. Integer-byte CA does not satisfy the real-valued premises exactly, especially after increments reach one byte.

8 Interpretation, scope, and reproducibility

What the transition explains. The exact reference gives a simple mechanism: when a group's required pacing time exceeds an RTT, reservations accumulate and the window multiplier is below two. Once it fits, ACK availability limits further growth and additional gain chiefly compresses transmission times. The finite threshold is $2D(2^m)$ and approaches $2 \ln 2$; it is not equal to that limit for every group. The instantaneous-rate model reaches the same limiting value through the flight constraint, despite predicting a different low-gain exponent.

What the SCP measurements establish. Retaining the trailing slot gives close agreement with the queued model's low-gain multiplier. The sample $g_c - 0.05$ has multiplier 1.950624, while the critical target has multiplier two at the selected boundaries. This brackets a finite observed change, not the implementation's exact infinite-time threshold. Figure 4 compares like-for-like packet-index groups; fixed-bin $L_n < R$ by construction cannot establish absence of group crossing. Neither native nor Python finite traces prove a universal CA critical gain.

Relation to existing work. Reno's slow-start update is specified in RFC 5681 [1]. Pacing's interaction with TCP dynamics has been studied well before this work [2]. RFC 9002 gives a cwnd/RTT-based pacing example with a multiplicative factor [3], and the IC-CRG pacing draft discusses practical pacing behavior, including compensation for slow-start growth [4]. The contribution here is a compact event-level solution for a particular reservation rule, together with a reproducible examination of how an actual integer implementation changes its predictions. It is not a claim to have first observed that insufficient pacing rate can throttle slow start.

Limits. The study isolates one flow, an initial window of one packet, fixed complete feedback delay, and separate loss-free SS and CA phases. The virtual network intentionally has no bottleneck serialization, propagation variation, cross traffic, delayed ACK policy, or receiver pressure. It therefore measures feedback and scheduling dynamics rather than performance on a real path. The sender executes genuine SCP code, but the environment remains an idealized virtual network. A different initial window, timer granularity, queue policy, or ACK policy can change finite

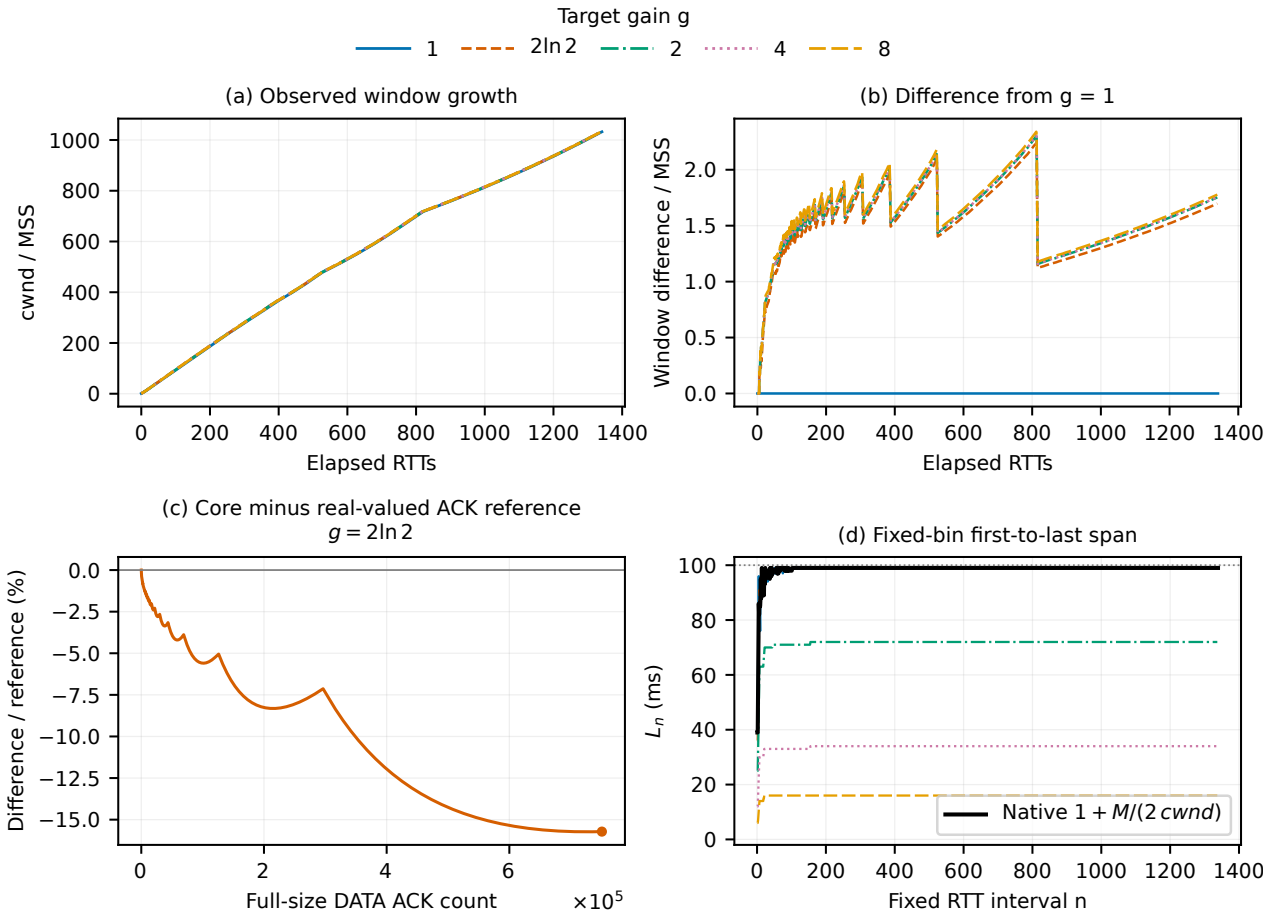


Figure 5: CA from one MSS, five representative fixed gains. (a) Observed RTT-boundary windows. (b) Differences from gain 1. (c) Critical-target window error against the real-valued recurrence at matching ACK counts, including the last full-packet ACK. (d) Actual fixed-interval spans, with the native adaptive policy in black; final-send intervals are excluded. Lines connect observations without fitting.

behavior. No conclusion here establishes an optimal Internet pacing gain, an improvement in fairness, or a universal CA window law in time.

Reproduction.

SS uses successful Actions run 39697564304 (`ef0076b`) and its branch guide. CA uses successful run 39697576064 (`9670889`) and its separate guide. Both share baseline `e41ac88`. Each branch stores its own compact plotting tables and analysis scripts. Large raw logs remain in Actions; the guides supply rerun commands after expiry. Python data are identified by frozen source hash and settings. Cloning and redrawing the compact figures require no raw archive or Release asset.

9 Conclusion

The ACK update alone does not determine growth in time. The immutable-reservation SS model has an exact gain-dependent solution; native transfers closely match its low-gain exponent while differing in phase and finite transition. In CA, gain changes ACK timing while integer-byte updates produce a substantial deficit against the real-valued ACK reference. In both campaigns, higher gain chiefly concentrates transmissions with little completion-time benefit. Evaluating pacing requires the window update rule, its integer representation, and actual packet timing.

References

[1] M. Allman, V. Paxson, and E. Blanton. *TCP Congestion Control.* RFC 5681, 2009. https://www.rfc-editor.org/rfc/rfc5681.

[2] A. Aggarwal, S. Savage, and T. Anderson. Understanding the performance of TCP pacing. *IEEE INFOCOM*, 2000, pp. 1157–1165.

[3] J. Iyengar and I. Swett. *QUIC Loss Detection and Congestion Control.* RFC 9002, Section 7.7, 2021. https://www.rfc-editor.org/rfc/rfc9002.

[4] M. Welzl et al. *Pacing in Transport Protocols*. Internet-Draft, draft-irtf-iccrg-pacing-02, 2026. Work in progress. https://datatracker.ietf.org/doc/html/draft-irtf-iccrg-pacing-02.

A Packet-Time Comparison and the Burst Limit

This appendix supplies the lower-comparison argument for the ideal reference scheduler in Section 3. Integer-time execution still requires separate packet-time comparison before transferring the theorem to an implementation. Both comparison systems use the same fixed feedback delay and one-MSS increase per ACK.

A.1 The Fixed-Batch Comparator Respects Flight

Let the preceding and current batch sizes be $p = F_{n+1}$ and $q = F_{n+2}$ in round $n \geq 1$. After phase $0 \leq u < 1$, its flight is

$$F(n+u) = p + \lfloor qu \rfloor - \lfloor pu \rfloor \leq q. \tag{19}$$

Its current window is at least q, so the comparator respects the actual-flight limit. It does not reserve all future flight positions at planning time: it can rely on ACKs arriving during the batch. This distinguishes its planning rule from the reference scheduler's invariant $F + Q = w$.

A.2 Every Reference Packet Leaves No Later at Unit Gain

At $g = 1$, $2D(q) \geq 1$ for all $q \geq 1$. Equation (7) reduces to $b_k = 2H_k - 1$, giving

$$s_0 = 0, \qquad s_{2k-1} = 2H_k - 1, \qquad s_{2k} = 2H_k - 1 + \frac{1}{k+1}. \tag{20}$$

The consecutive spacings decrease with packet index:

$$s_{j+1} - s_j = \frac{1}{\lfloor (j+1)/2 \rfloor + 1}, \qquad j \geq 0. \tag{21}$$

The fixed batch of size q starts with packet index $J_n = p + q - 2$. Its timestamps are

$$\widehat{s}_j = n + \frac{j - J_n}{q}, \qquad J_n \leq j < J_n + q. \tag{22}$$

Within this batch, the successive differences of $s_j - \widehat{s}_j$ are the decreasing spacings above minus $1/q$. A maximum is attained at $j_* = 2q-3$. This index belongs to the batch because $j_* - J_n = q-p-1 \geq 0$ and $J_n + q - 1 - j_* = p \geq 1$. Substitution yields

$$\max_{J_n \leq j < J_n + q} (s_j - \widehat{s}_j) = -E_n, \qquad E_n = n + 2 - \frac{p-1}{q} - 2H_q. \tag{23}$$

Initially $p = 1, q = 2$, and $E_1 = 0$. In the next batch, (p, q) becomes $(q, p+q)$, so

$$\begin{aligned} E_{n+1} - E_n &= 1 - \frac{q-1}{p+q} + \frac{p-1}{q} - 2\sum_{i=1}^{p} \frac{1}{q+i} \\ &= 2\sum_{i=1}^{p} \frac{i(p-i)}{q(q+p)(q+i)} \geq 0. \end{aligned} \tag{24}$$

For the second equality, subtract $1/(q+i)$ from $(q+p-i)/[q(q+p)]$; the difference is $i(p-i)/[q(q+p)(q+i)]$. Summing these identities gives the second line of (24). Thus $E_n \geq 0$, proving $s_j \leq \widehat{s}_j$ in every batch. Packet zero is common to both systems. Earlier corresponding sends give earlier corresponding ACKs, hence no smaller window at any observation time.

A.3 Extension to Larger Gains

For $g_2 \geq g_1 \geq 1$, induction over ACK batches gives $s_j(g_2) \leq s_j(g_1)$. The initial send is common. If earlier packet times are ordered, their fixed-delay ACK times are ordered. The previous reservation tail is also ordered, since every newly added spacing decreases as gain increases. Taking the maximum of the ACK time and the previous tail preserves the order, and so does adding the second packet's spacing.

Combining this monotonicity with the unit-gain comparison and the flight upper bound proves, specifically for (3),

$$MF_{n+2} \leq \mathrm{cwnd}_g(nR) \leq M2^n, \qquad g \geq 1. \tag{25}$$

The actual-time comparison is essential. Merely appending more packets earlier does not order their departures under arbitrary queue or timestamp rules.

A.4 Approaching Burst Sending

For $g \geq g_c$, $B_m = m + 1$. For any fixed packet index j in group m, its timestamp therefore has the form $s_j = m + 1 + d_j/g$, with $d_j \geq 0$ independent of gain. Including packet zero,

$$\lim_{g \to \infty} s_j(g) = \lfloor \log_2(j+1) \rfloor. \tag{26}$$

This is the immediate-send sequence: one packet at time zero, two at one RTT, four at two RTTs, and so on. For finite gain, packets remain locally spaced, yet their group occupies a shrinking fraction of an RTT. Uniform spacing within a group therefore coexists with growing concentration across time.

There is a boundary subtlety. At finite high gain, $W_n = 1 + 2^{n-1}$, whereas the immediate-send window is 2^n. ACKs just after a boundary coalesce onto that boundary as gain tends to infinity. Counting events at a jump is not continuous under convergence of their timestamps. The packet-time burst limit is valid without asserting pointwise convergence of the window at every integer RTT.